Gilberto Hernández

Fecha: 20/1/2026

Music Player App

Sistema Web de Streaming Musical

ÍNDICE

1. INTRODUCCIÓN
1.1 Presentación General del Proyecto
1.2 Justificación del Tema Elegido
1.3 Objetivos
    1.3.1 Objetivo General
    1.3.2 Objetivos Específicos

2. DESCRIPCIÓN DEL PROBLEMA
2.1 Contexto del Problema Seleccionado
2.2 Aplicación en un Escenario Real

3. DISEÑO DE LA SOLUCIÓN
3.1 Arquitectura General y Flujo del Proceso
3.2 Herramientas y Tecnologías Utilizadas

4. IMPLEMENTACIÓN TÉCNICA
4.1 Modelos y Servicios
4.2 Frontend, Backend y Justificación Técnica

5. EVALUACIÓN DE DESEMPEÑO

6. CONCLUSIONES
6.1 Aprendizajes Individuales
6.2 Retos Enfrentados y Superados

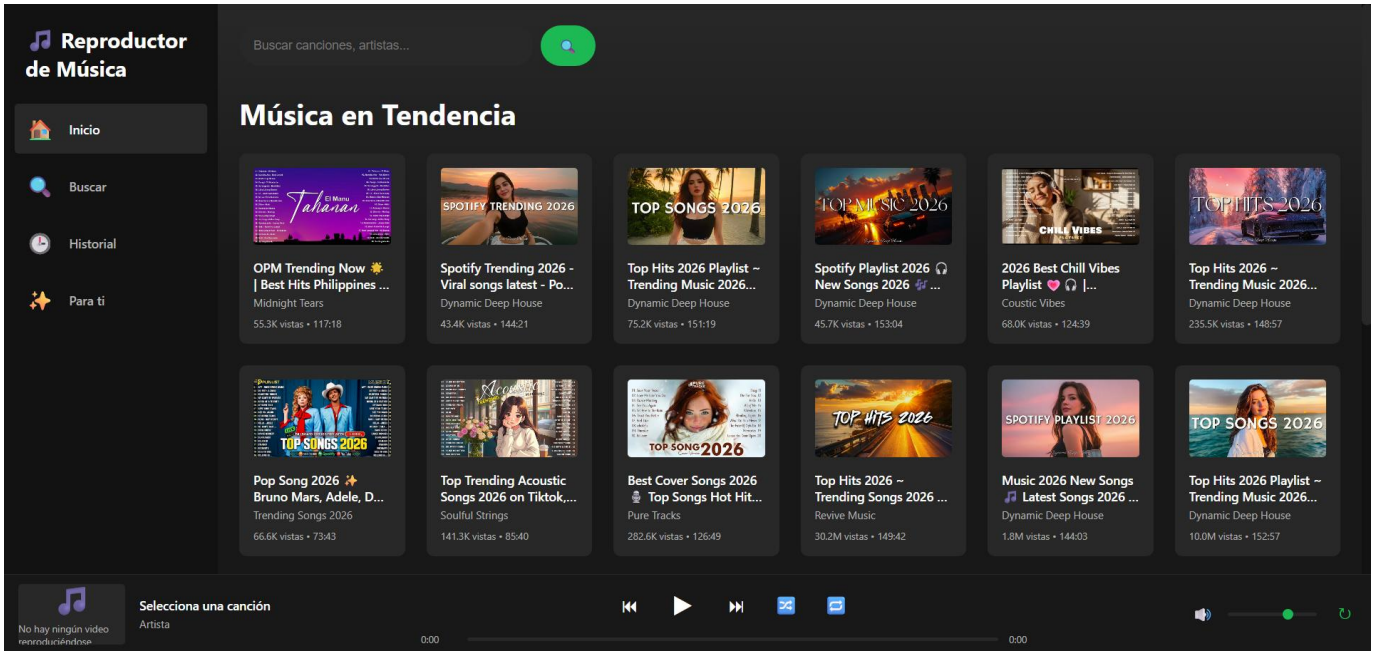
7. ANEXOS
A. Manual del Sistema
  A.1 Requisitos del Sistema
  A.2 Estructura del Proyecto
  A.3 Instalación del Proyecto
  A.4 Ejecución del Servidor
  A.5 Uso del Sistema
  A.6 Base de Datos
  A.7 Personalización
  A.8 Recomendaciones Finales

1. INTRODUCCIÓN

1.1 Presentación General del Proyecto

El presente documento describe el diseño, desarrollo e implementación de un Sistema Web de Streaming Musical desarrollado bajo una arquitectura cliente-servidor utilizando Python como tecnología principal en el backend. La aplicación permite la reproducción de contenido multimedia mediante una interfaz web interactiva que integra audio y video de forma sincronizada, ofreciendo al usuario una experiencia similar a las plataformas modernas de distribución musical.

El sistema fue desarrollado con el objetivo de aplicar conocimientos de desarrollo full-stack, combinando Flask como framework web, JavaScript para la manipulación dinámica del DOM y control del elemento multimedia HTML5, así como SQLite como sistema gestor de base de datos ligera. La aplicación permite buscar canciones, reproducir contenido, controlar volumen, alternar modos de reproducción y gestionar un historial básico.



Este proyecto representa una integración completa entre frontend y backend, priorizando control preciso de eventos asincrónicos y estabilidad en la reproducción multimedia.

1.2 Justificación del Tema Elegido

El desarrollo de aplicaciones multimedia web representa uno de los escenarios más exigentes dentro del desarrollo de software moderno, debido a la necesidad de sincronizar múltiples eventos en tiempo real y garantizar estabilidad en la experiencia del usuario. La elección de este proyecto responde al interés de profundizar en el manejo programático del elemento <video> de HTML5 y en la gestión de estados dinámicos mediante JavaScript.

Además, se eligió Python como lenguaje principal del backend debido a su versatilidad, claridad sintáctica y amplia adopción en entornos profesionales. Implementar una aplicación de streaming musical permite demostrar competencias en arquitectura web, diseño de APIs REST, manipulación del DOM y manejo de asincronía, elementos fundamentales para un desarrollador de software orientado a soluciones modernas.

El proyecto también fortalece habilidades en diseño de experiencia de usuario y control de flujo de datos entre cliente y servidor.

1.3 Objetivos

1.3.1 Objetivo General

Desarrollar un sistema web de streaming musical utilizando Python y Flask como backend, integrando una interfaz interactiva que permita la reproducción y control dinámico de contenido multimedia.

1.3.2 Objetivos Específicos

- Implementar una arquitectura cliente-servidor funcional.
- Desarrollar un reproductor multimedia.
- Gestionar estados de reproducción (play, pause, siguiente, anterior).
- Integrar una base de datos SQLite para almacenamiento básico.
- Diseñar una interfaz moderna inspirada en plataformas actuales.
- Garantizar sincronización estable entre video y controles visuales.
- Documentar técnicamente la solución implementada.

2. DESCRIPCIÓN DEL PROBLEMA

2.1 Contexto del Problema Seleccionado

Las aplicaciones de reproducción multimedia en entornos web presentan desafíos técnicos relacionados con la sincronización de eventos, control del tiempo de reproducción y gestión del estado interno del reproductor. Cuando se trabaja con el elemento <video> de HTML5, es común enfrentar problemas como reinicios inesperados, desincronización entre la barra de progreso y el tiempo real, conflictos entre eventos timeupdate e input, y errores derivados de valores no válidos como NaN en la duración del contenido.

Estos inconvenientes afectan directamente la experiencia del usuario y requieren una implementación cuidadosa para garantizar estabilidad y precisión en el comportamiento del sistema.

2.2 Aplicación en un Escenario Real

En un entorno real, plataformas de distribución musical y audiovisual deben garantizar que el usuario pueda reproducir contenido sin interrupciones y visualizar correctamente el progreso de reproducción. Si el sistema no maneja adecuadamente los eventos asincrónicos, el usuario experimenta comportamientos inconsistentes.

Este proyecto simula ese escenario real, enfocándose en resolver conflictos comunes en reproductores personalizados desarrollados desde cero, sin depender de librerías externas especializadas.

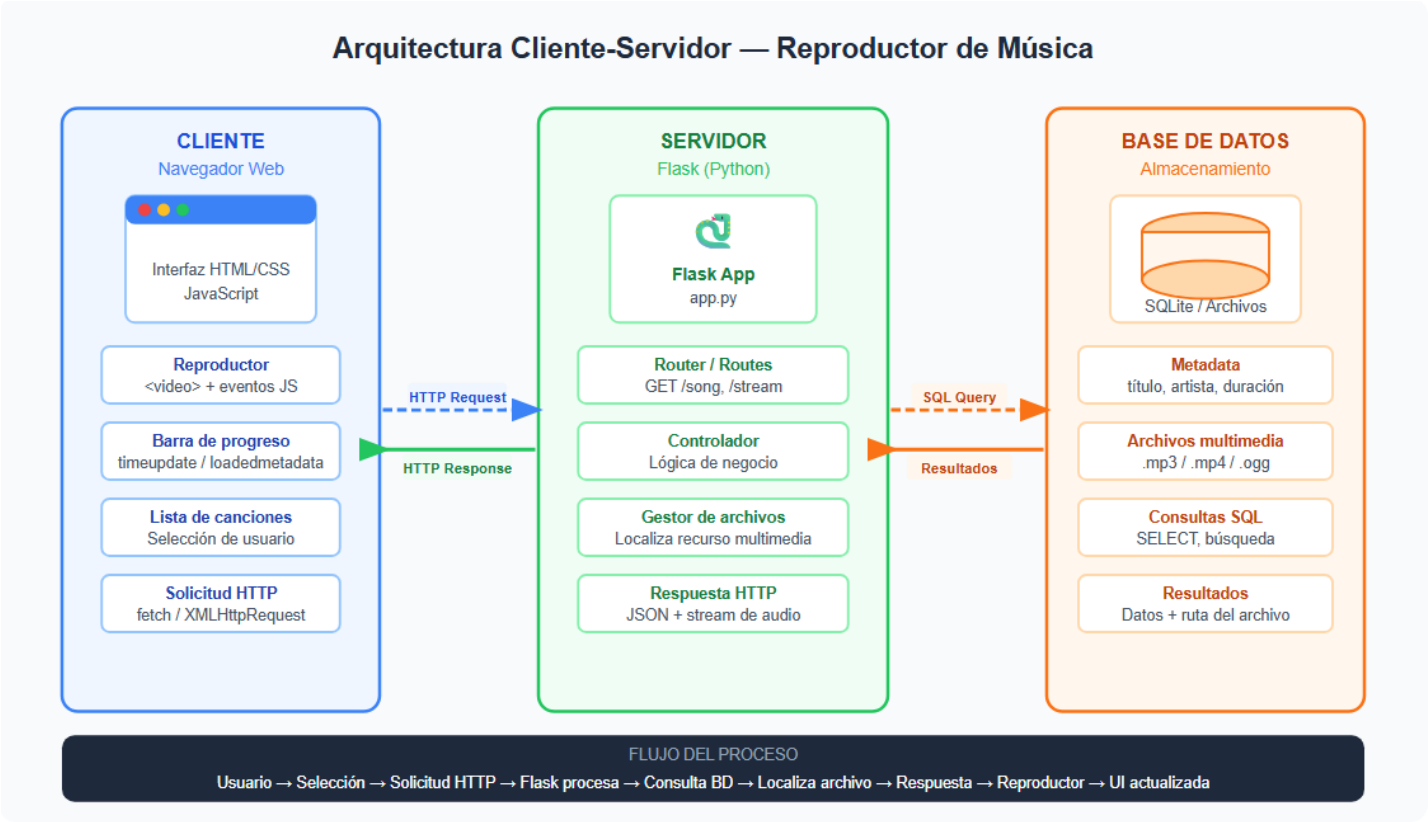
3. DISEÑO DE LA SOLUCIÓN

3.1 Arquitectura General y Flujo del Proceso

La solución fue diseñada bajo una arquitectura cliente-servidor tradicional, donde el frontend se encarga de la interacción con el usuario y el backend gestiona las solicitudes, acceso a datos y entrega de recursos multimedia. El navegador actúa como cliente, ejecutando scripts JavaScript que controlan dinámicamente el comportamiento del reproductor y la interfaz.

Cuando el usuario selecciona una canción, el sistema ejecuta una solicitud HTTP hacia el backend desarrollado en Flask. Este procesa la petición, localiza el recurso multimedia correspondiente y responde con la información necesaria para su reproducción. El frontend asigna la fuente al elemento <video>, inicializa los eventos loadedmetadata y timeupdate, y actualiza la interfaz visual.

El flujo completo es el siguiente:



Este diseño separa responsabilidades, permitiendo modularidad y futura escalabilidad.

3.2 Herramientas y Tecnologías

El backend fue desarrollado en Python utilizando el framework Flask por su ligereza y facilidad de integración. La base de datos SQLite fue seleccionada por su simplicidad y portabilidad. En el frontend se emplearon HTML5 y CSS3 para la estructura y diseño visual, mientras que JavaScript fue utilizado para la manipulación dinámica del DOM y control del reproductor multimedia.

Estas tecnologías permiten una solución completa full-stack sin depender de frameworks complejos adicionales.

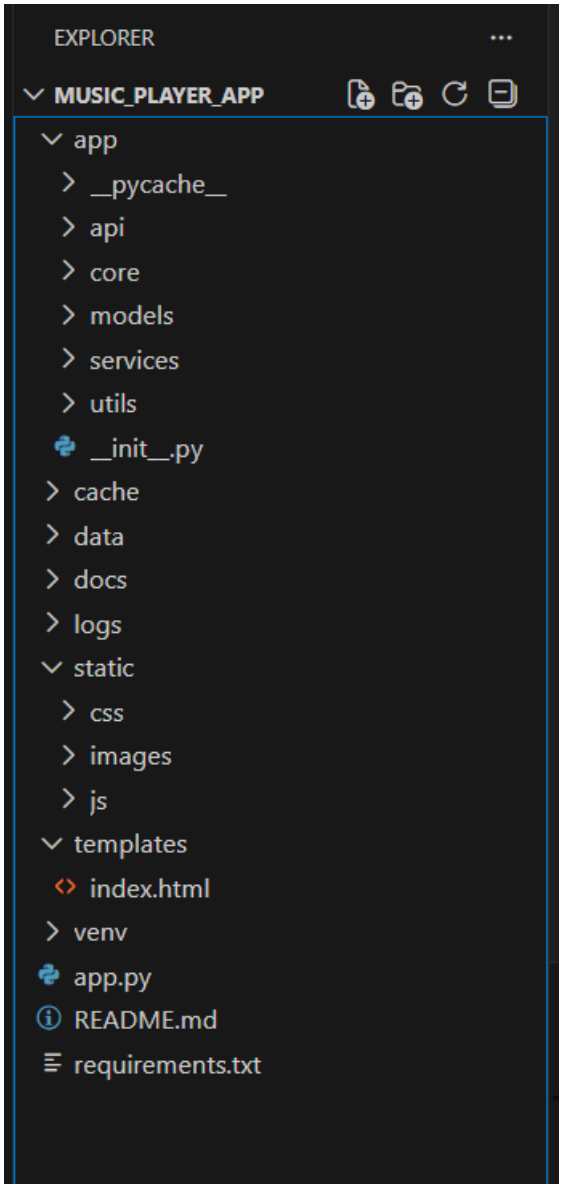
4. IMPLEMENTACIÓN TÉCNICA

4.1 Modelos y Servicios

La implementación técnica del sistema se estructuró en una organización modular que separa claramente la lógica del servidor, el acceso a datos y la interacción con el cliente. En el backend, desarrollado con Flask, se definieron rutas específicas que actúan como servicios REST encargados de responder solicitudes relacionadas con la búsqueda de canciones, carga de recursos multimedia y registro básico de historial.

El modelo principal corresponde a la entidad de contenido multimedia, la cual almacena información como identificador interno, nombre de la canción, artista o canal asociado y referencia al archivo físico almacenado en el servidor. Estos datos se gestionan mediante SQLite, lo que permite consultas rápidas sin necesidad de configurar un servidor de base de datos externo.

El servicio de reproducción no transmite el archivo de manera fragmentada tipo streaming avanzado, sino que entrega la ruta controlada al cliente, quien gestiona la reproducción mediante el elemento <video> de HTML5. Esta decisión simplifica la arquitectura sin comprometer la funcionalidad principal del sistema.



Además, se implementaron validaciones básicas para evitar solicitudes inválidas y garantizar que solo se acceda a archivos previamente registrados en el sistema.

4.2 Controladores y Código Clave

Los controladores en Flask manejan las rutas principales del sistema. Cada endpoint recibe solicitudes HTTP y retorna respuestas en formato JSON cuando corresponde. Por ejemplo, al seleccionar una canción, el controlador consulta la base de datos, obtiene la información necesaria y responde con los datos estructurados que el frontend utilizará para actualizar la interfaz.

En el lado del cliente, el código JavaScript cumple un rol central. Se implementaron eventos como loadedmetadata, que permite obtener la duración real del archivo multimedia, y timeupdate, que actualiza constantemente la barra de progreso y el tiempo visual mostrado al usuario.

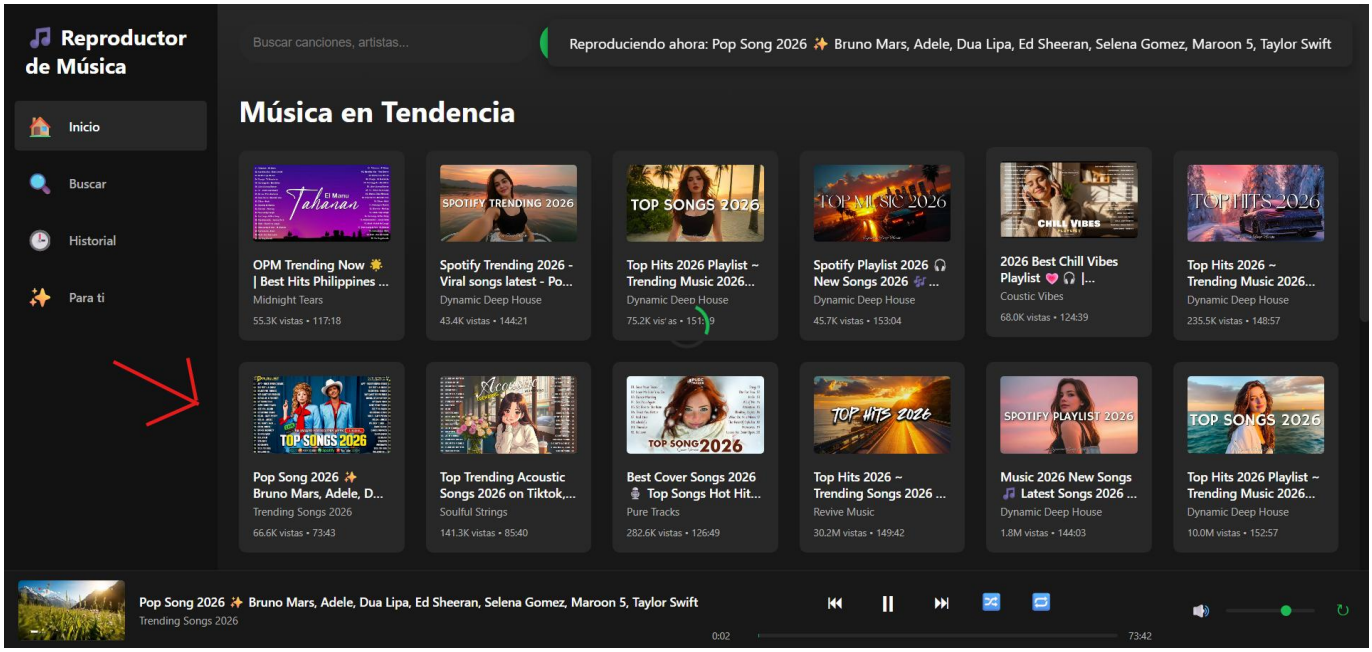
Un aspecto clave fue la implementación de una variable de control para evitar conflictos entre la actualización automática del progreso y la manipulación manual del usuario al arrastrar el slider.

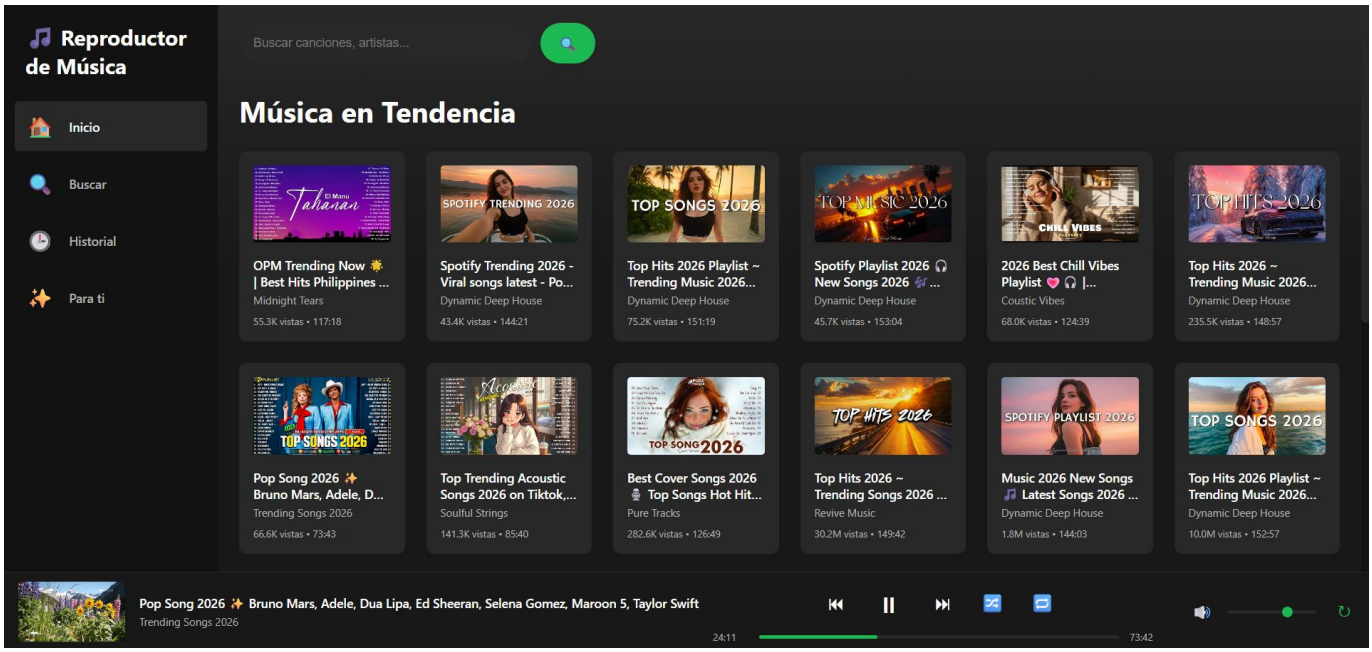
4.3 Frontend, Backend y Justificación Técnica

El frontend fue diseñado como una interfaz dinámica que responde inmediatamente a las acciones del usuario. La barra de progreso personalizada se vincula directamente al atributo currentTime del elemento <video>, permitiendo adelantar o retroceder sin reiniciar la reproducción. Para evitar errores comunes, se validó que la duración del video estuviera correctamente cargada antes de realizar cálculos de porcentaje.

El backend cumple la función de intermediario seguro entre los archivos multimedia y el cliente, evitando exponer rutas sensibles del sistema. Esta separación refuerza la arquitectura y permite futuras mejoras como autenticación o control de acceso.

La decisión de utilizar Flask en lugar de frameworks más complejos se basó en la necesidad de mantener un proyecto académico claro, comprensible y fácilmente documentable. Por su parte, JavaScript puro fue seleccionado para demostrar dominio del control manual de eventos sin depender de librerías externas.



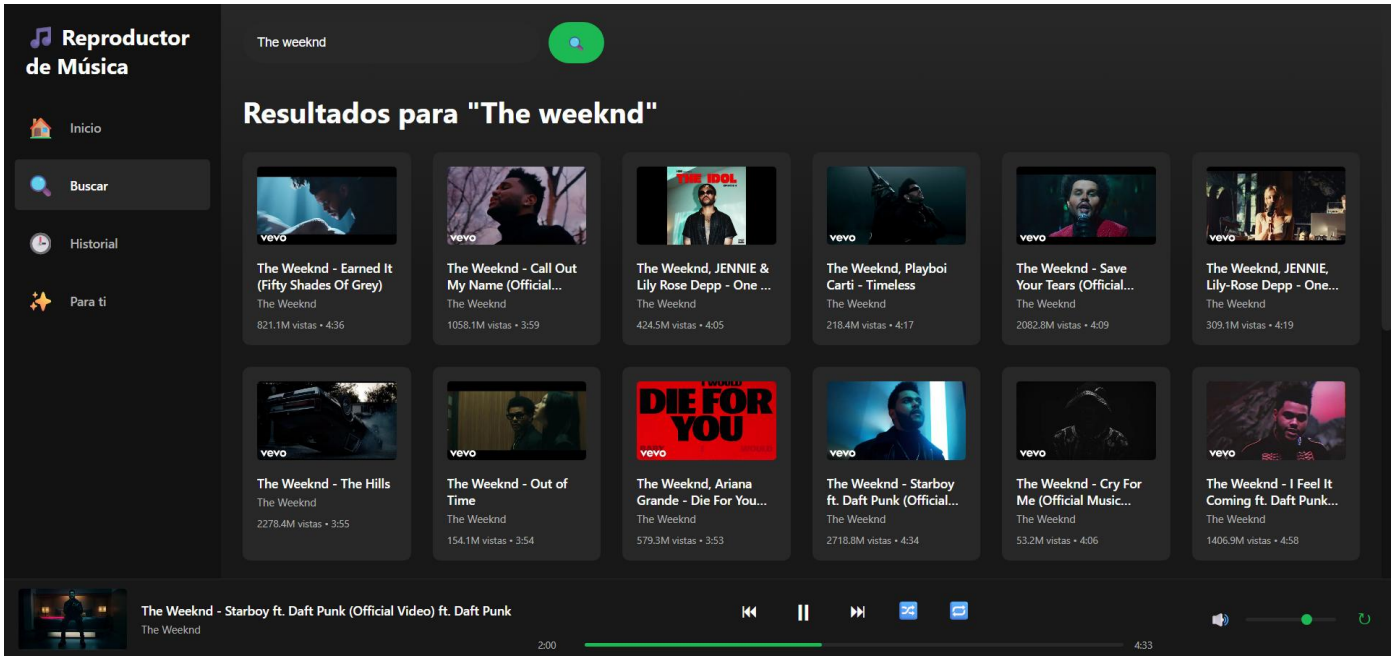


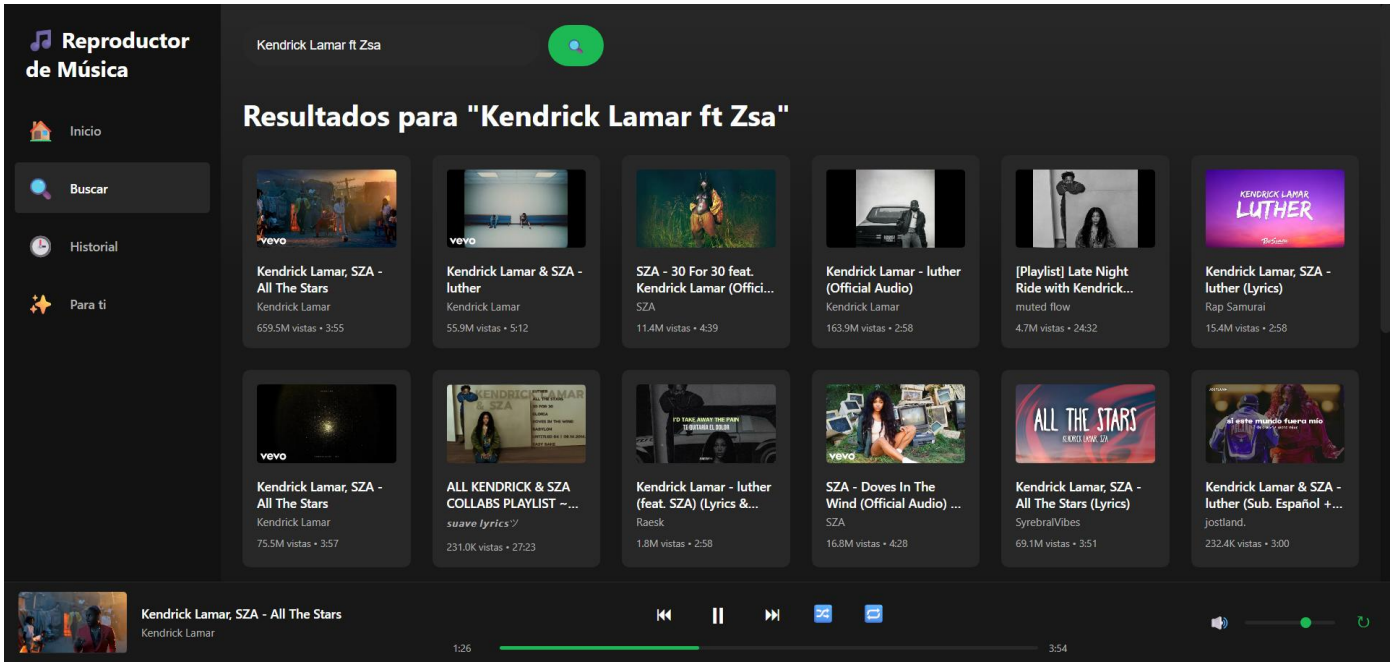
La implementación final logra una experiencia estable, coherente y técnicamente justificada.

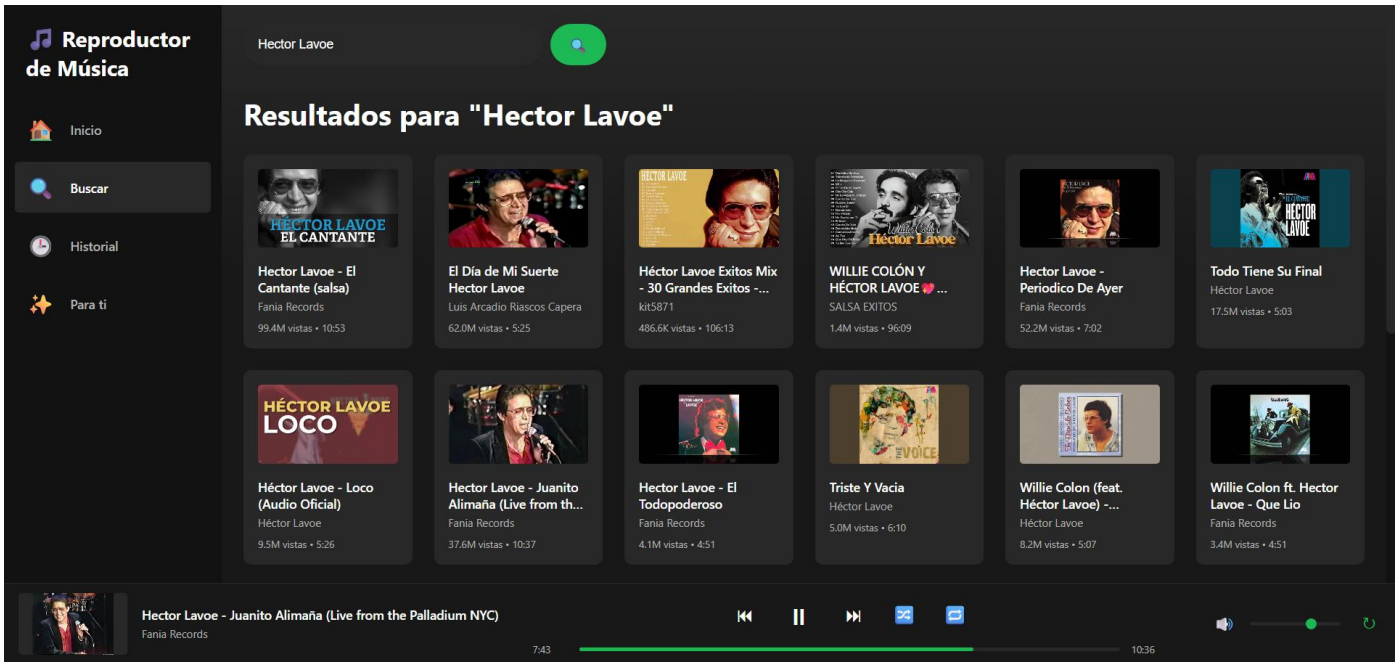
5. EVALUACIÓN DE DESEMPEÑO

Se realizaron pruebas de reproducción con archivos multimedia de distinta duración para verificar estabilidad en la sincronización del progreso. El sistema mantuvo comportamiento consistente sin desincronización entre tiempo visual y reproducción real.

Imágenes de ejemplo:







6. CONCLUSIONES

6.1 Mi Aprendizaje

El desarrollo de este sistema de streaming musical representó una experiencia de aprendizaje significativa en el ámbito del desarrollo web full-stack. A nivel técnico, fortalecí mis conocimientos en la construcción de arquitecturas cliente-servidor utilizando Python y Flask, comprendiendo con mayor profundidad la importancia de separar responsabilidades entre frontend y backend.

Uno de los principales aprendizajes fue el manejo del elemento <video> de HTML5 y la gestión de sus eventos asociados. Aprendí también a trabajar con estados internos que permiten evitar comportamientos inesperados, como desincronización de la barra de progreso.

Asimismo, reforcé mis habilidades lo cual me permitió entender con mayor claridad la lógica detrás de frameworks modernos. Este proyecto me ayudó a consolidar mi capacidad de análisis, depuración de errores y toma de decisiones técnicas fundamentadas.

6.2 Retos Enfrentados y Superados

Durante el desarrollo surgieron diversos desafíos técnicos que requirieron análisis y pruebas constantes. Uno de los principales retos fue manejar correctamente los valores de duración del video cuando aún no estaban completamente cargados, lo que ocasionaba errores de tipo NaN en los cálculos del porcentaje de progreso. Este problema se resolvió implementando validaciones previas antes de realizar operaciones matemáticas.

También resultó complejo mantener la sincronización visual entre la interfaz y el estado real del reproductor. Sin embargo, mediante pruebas iterativas y control cuidadoso de variables de estado, se logró una solución estable.

Superar estos retos fortaleció mi comprensión del desarrollo orientado a eventos y me permitió implementar una solución sólida y funcional.

7. ANEXOS

Manual del Sistema y Guía de Configuración

1. Requisitos del Sistema

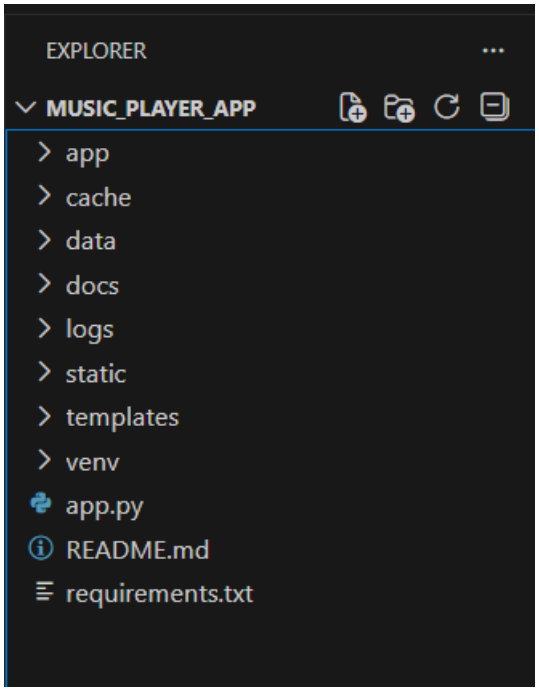
Para ejecutar correctamente la aplicación de streaming musical, se deben cumplir los siguientes requisitos mínimos:

- Python 3.10 o superior instalado.
- Python 3.11.9 (necesario para el entorno virtual)
- Gestor de paquetes pip.
- Navegador web moderno (Google Chrome, Microsoft Edge o Firefox).
- Sistema operativo Windows (recomendado)
- Editor de código (Visual Studio Code recomendado).

El sistema no requiere servidor externo de base de datos, ya que utiliza SQLite como motor integrado.

2. Estructura del Proyecto

Una vez clonado el repositorio, la estructura principal del proyecto es la siguiente:



Archivo raíz

- **app.py**: Archivo principal que ejecuta el servidor Flask. Es el punto de entrada de la aplicación.
- **requirements.txt**: Contiene las dependencias necesarias del proyecto (Flask y otras librerías).
- **README.md**: Documento con información general del proyecto, instalación y uso.

Carpeta app/

Contiene la lógica principal organizada bajo una arquitectura modular.

- **__init__.py**: Inicializa la aplicación Flask y configura los módulos.

api/

- **routes.py**: Define las rutas (endpoints) de la API.
- **__init__.py**: Inicializa el módulo de rutas.

core/

Contiene configuraciones y utilidades base del sistema.

- **config.py**: Configuración general del proyecto.
- **cache.py**: Manejo de almacenamiento en caché.
- **logger.py**: Configuración del sistema de logs.

models/

- **database.py**: Configuración y conexión a la base de datos SQLite.
- **__init__.py**: Inicializa los modelos.

services/

Contiene la lógica de negocio.

- **youtube_search.py**: Servicio para buscar música en YouTube.
- **video_stream.py**: Manejo del streaming de video.
- **recommendations.py**: Sistema de recomendaciones musicales.
- **performance.py**: Monitoreo o métricas de rendimiento.

utils/

- **utils.py**: Funciones auxiliares generales.
- **validators.py**: Validaciones de datos.

Carpeta templates/

Contiene los archivos HTML renderizados por Flask.

- **index.html**: Vista principal del reproductor de música.

Carpeta static/

Recursos estáticos del frontend.

css/

- **style.css**: Estilos de la aplicación.

js/

- **app.js**: Lógica del frontend en JavaScript.

images/

- **placeholder.jpg**: Imagen por defecto utilizada en la interfaz.

Carpeta data/

- **music_player.db**: Base de datos SQLite que almacena la información del sistema.

Carpeta cache/

Contiene archivos .cache generados automáticamente para optimizar búsquedas o consultas externas (por ejemplo, resultados de YouTube).

Carpeta logs/

Almacena los registros del sistema.

- **app.log**: Log general de la aplicación.
- **error.log**: Registro de errores.
- **app.json.log**: Log estructurado en formato JSON.

Carpeta venv/

Entorno virtual de Python con todas las dependencias instaladas.
(No es recomendable subir esta carpeta a un repositorio; normalmente se excluye con .gitignore).

3. Instalación del Proyecto

Paso 1: Clonar el Repositorio

Abrir una terminal y ejecutar:

```
git clone https://github.com/GilbertoHernandez150/Portafolio/tree/main/Python/Music_Player_App
```

Luego ingresar al directorio:

```
cd Music_Player_App
```

Paso 2: Entorno Virtual

Crear entorno:

- py -3.11 -m venv venv

Activar entorno:

Windows:

- venv\Scripts\activate

Mac/Linux:

- source venv/bin/activate

Paso 3: Instalar Dependencias

pip install flask

pip install -r requirements.txt

4. Ejecución del Servidor

Para iniciar la aplicación:

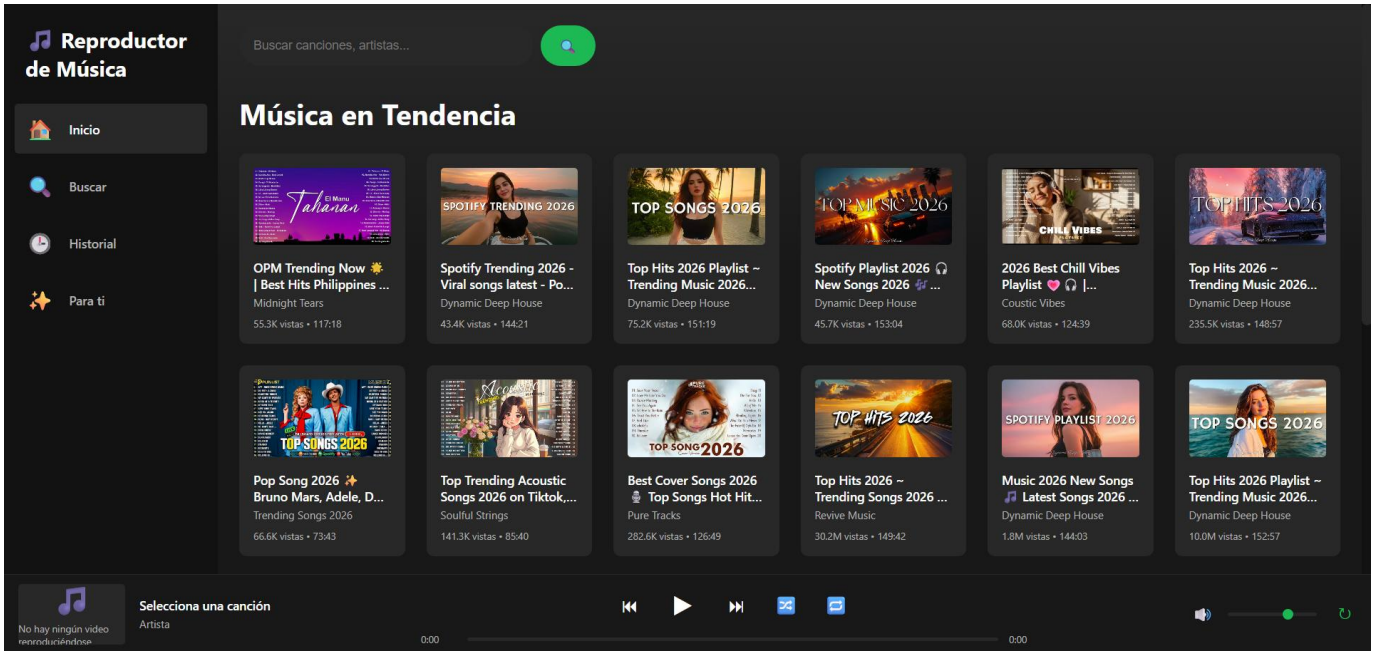
- python app.py

El servidor se ejecutará en:

- http://127.0.0.1:5000

Abrir esa dirección en el navegador.

Imagen de muestra



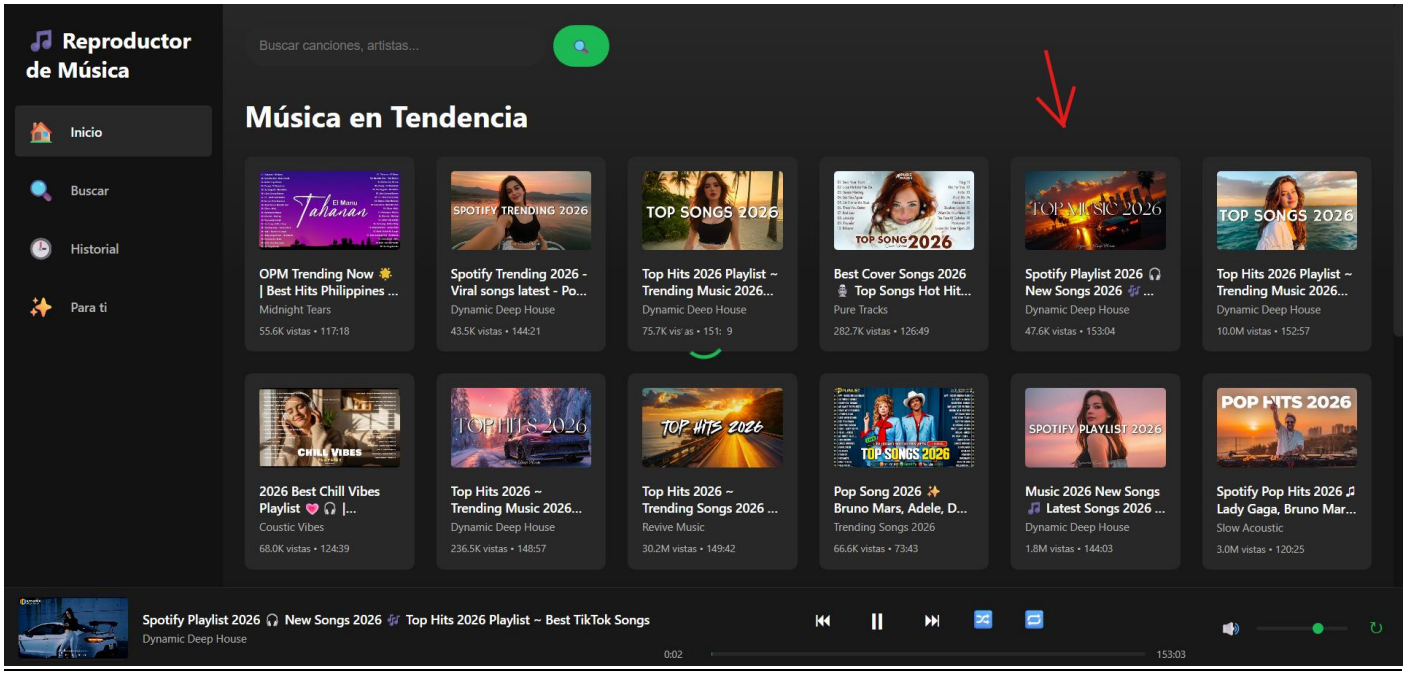
5. Uso del Sistema

5.1 Selección de Canción

Al ingresar a la aplicación, se muestra la lista de canciones disponibles. El usuario puede hacer clic sobre cualquiera para iniciar la reproducción.

Al seleccionar una canción:

- Se envía una solicitud al backend.
- Se obtiene la información del archivo multimedia.
- El reproductor inferior se activa.
- Se actualiza el título y artista.
- El contenido comienza a reproducirse automáticamente.



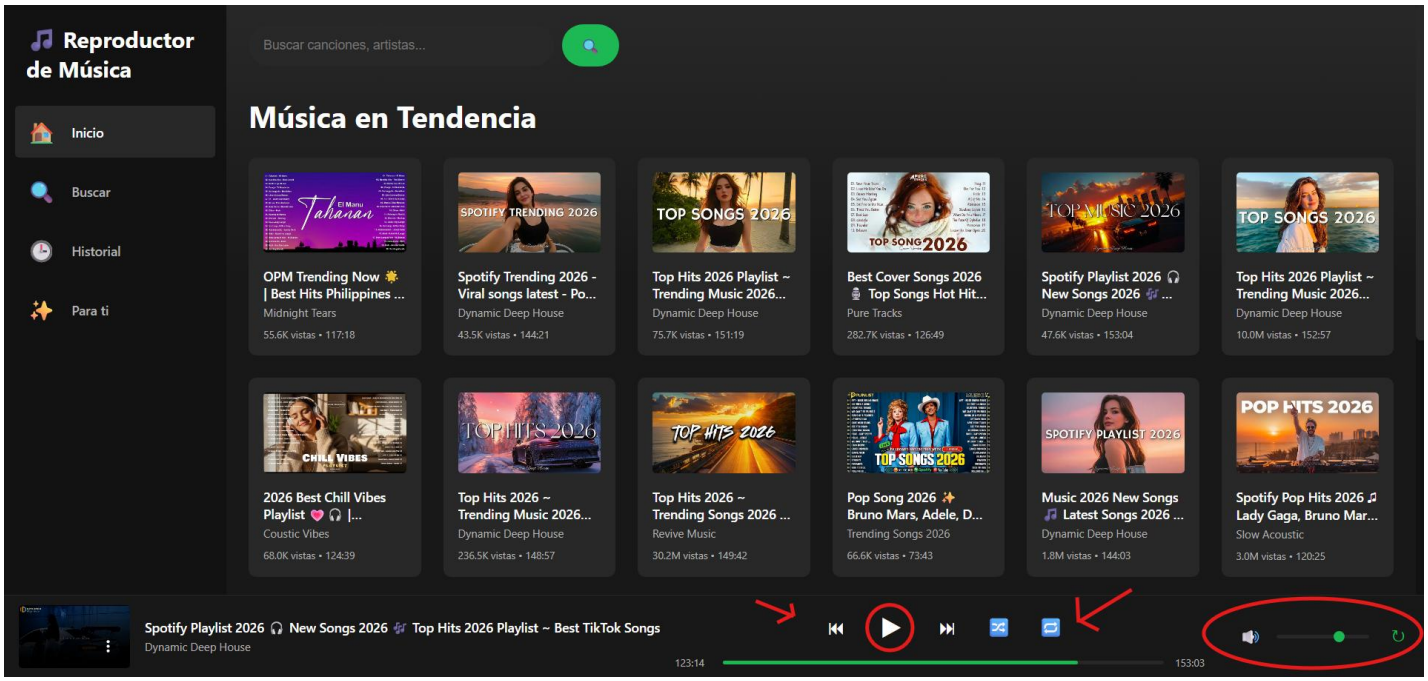
5.2 Controles de Reproducción

El panel inferior incluye:

- Botón Play/Pause.
- Botón Siguiente.
- Botón Anterior.
- Botón Shuffle.
- Botón Repeat.
- Control de volumen.
- Barra de progreso interactiva.

El usuario puede:

- Pausar y reanudar reproducción.
- Ajustar volumen dinámicamente.
- Adelantar manualmente la canción.
- Retroceder a un punto específico.



6. Base de Datos

La base de datos SQLite se encuentra en:

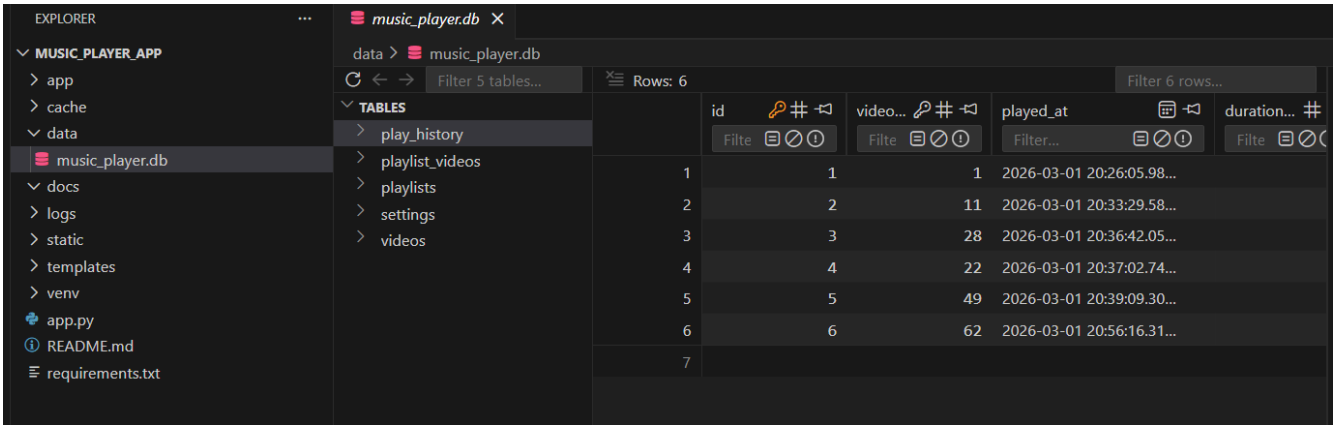
data/music_player.db

Incluye tabla principal para almacenar:

- Identificador.
- Nombre de canción.
- Artista.
- Ruta del archivo.

Para visualizar la base de datos puede utilizarse:

- SQLite Viewer.



7. Personalización

El sistema puede modificarse fácilmente para:

- Cambiar estilos CSS.
- Integrar autenticación.
- Implementar playlists.
- Migrar base de datos a PostgreSQL.

9. Recomendaciones Finales

Se recomienda:

1. Ejecutar en entorno virtual.
2. No modificar directamente archivos multimedia en producción.
3. Mantener respaldos de la base de datos.

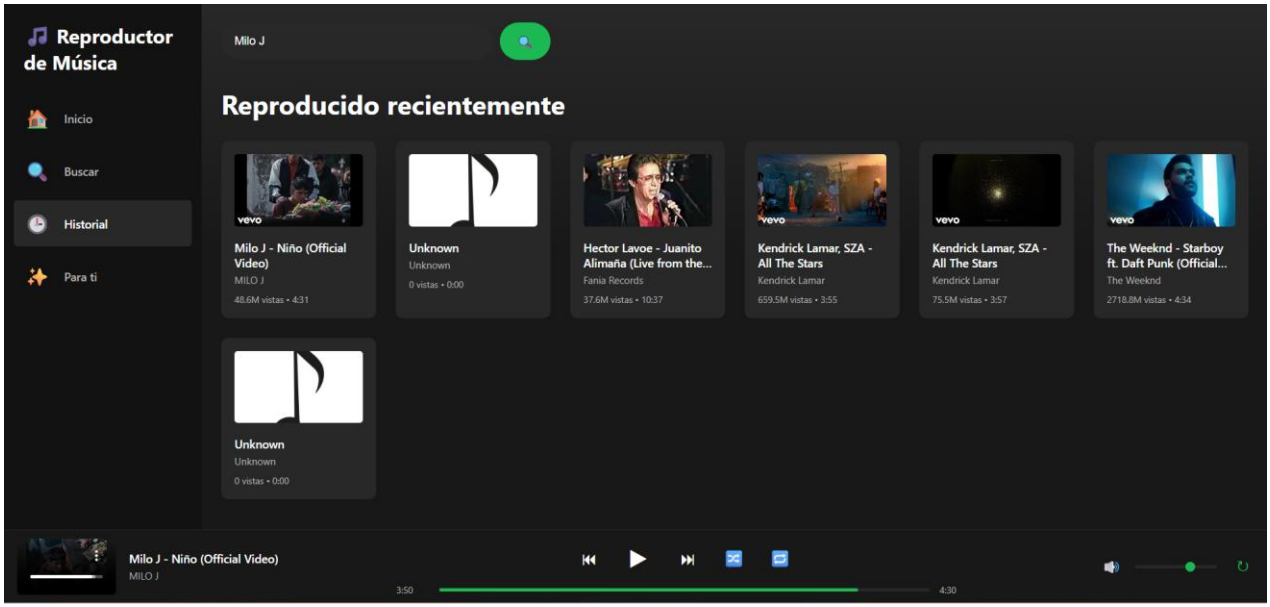



Imagen del historial con las canciones mostradas en el documento.